

# Exercises

## Declaring Types, Trees

### Exercise 1:

Looking at the following definition of *tail* :

```
tail :: [a] -> [a]
tail [] = []
tail (_:xs) = xs
```

Rewrite this (call it *safetail*) using:

1. (*safetailMaybe*) Maybe (returning Nothing when called on empty list)
2. (*safetail*”) Either (returning error message when called on empty list)

### Exercise 2:

Referring to the abstract machine written in class notes:

— *Haskell Code for Abstract Machine example, Hutton, Chapter 7/*

```
data Expr = Val Int | Add Expr Expr | Mult Expr Expr
```

```
data Op = EVAL Expr | ADD Int | MULT Int
```

```
type Cont = [Op]
```

```
eval :: Expr -> Cont -> Int
eval (Val n)      c = exec c n
eval (Add x y)    c = eval x (EVAL y: c)
```

```
exec :: Cont -> Int -> Int
exec []          n = n
exec (EVAL y: c) n = eval y (ADD n: c)
exec (ADD n : c) m = exec c (n + m)
```

```
val :: Expr -> Int
val e = eval e []
```

Write out the evaluation of the following Expressions

1. Val 1
2. Add (Val 1) (Val 2)

3.        (Add (Add (Val 2) (Val 3) ) (Val 4))

**Exercise 3:**

The abstract machine (as per above) only implements *addition*. Show how you would extend this implementation to implement multiplication.

**Exercise 4:**

(Using the Nat example from earlier)

In a similar manner to the function *add*, define a recursive multiplication function

`mult :: Nat -> Nat -> Nat`

for the recursive type of natural numbers.

**Hint:** Make use of *add* in your definition

**Exercise 5:**

Using the following (as seen in class) :

`data Ordering = LT | EQ | GT`

together with a function

`compare :: Ord a => a -> a -> Ordering`

that decides if one value of an ordered type is less than (LT), equal to (EQ), or greater than (GT) another value. Using this function, redefine the function

`occurs :: Ord a => a -> Tree a -> Bool`

for search trees. Why is this new definition more efficient than the original version?

**Exercise 6:**

Consider the following type of binary trees:

`data Tree a = Leaf a | Node (Tree a) (Tree a)`

Let us say that such a tree is *balanced* if the number of leaves in the left and right subtree differs by at most one, with the leaves themselves being trivially balanced.

1. Define a function *leaves* that returns the number of leaves in a tree.

`leaves :: Tree a -> Int`

2. Using *leaves* above, or otherwise, define a function *balanced* that decides if a tree is balanced or not.

`balanced :: Tree a -> Bool`

3. Define a function ***depth*** that calculates the depth of a tree, where the depth is given by the number of nodes in the longest path from the root of the tree to a leaf in the tree.

```
depth :: Tree a -> Int
```

**Exercise 7:**

Define a function

```
balance :: [a] -> Tree a
```

that converts a non-empty list into a balanced tree.

***Hint:*** first define a function that splits a list into two halves whose length differs by at most one.

**Exercise 8:**

Using the idea of the search tree used in class with a slight change,

```
data Tree a =
    EmptyTree
  | Node (Tree a) a (Tree a) deriving (Show, Read, Eq)
```

```
occurs x (Node l y r) | x == y = True
                      | x < y  = occurs x l
                      | x > y  = occurs x r
```

```
treeInsert :: (Ord a) => a -> Tree a -> Tree a
```

— *Write the code for this*

```
flatten :: Tree a -> [a]
```

— *Write the code for this*

**Exercise 9:**

Define appropriate versions of the library functions:

1. **repeat** ::  $a \rightarrow [a]$   
**repeat**  $x = xs$  **where**  $xs = x:xs$
  
2. **take** ::  $\mathbf{Int} \rightarrow [a] \rightarrow [a]$   
**take** 0  $-$   $= []$   
**take**  $-$   $[] = []$   
**take**  $n$   $(x:xs)$   $= x : \mathbf{take} (n - 1) xs$
  
3. **replicate** ::  $\mathbf{Int} \rightarrow a \rightarrow [a]$   
**replicate**  $n = \mathbf{take} n . \mathbf{repeat}$
  
4. **map** ::  $(a \rightarrow b) \rightarrow [a] \rightarrow [b]$   
**map**  $-$   $[] = []$   
**map**  $f$   $(x:xs)$   $= f\ x : \mathbf{map} f xs$

for the following type of binary trees:

```
data Tree a = Leaf | Node a (Tree a) (Tree a)
           deriving Show
```

You should test it on the following test Tree (or similar)

```
myTree :: Tree Int
myTree = Node 1 (Node 6 (Node 4 (Leaf) (Leaf)) (Leaf) )
              (Node 3 Leaf Leaf )
```